



UNIVERSITY OF BERGEN

Transformer Models

Encoder-only Sentiment Classification and Decoder-only Text Generation

INF265 - Deep Learning

Project 3

Andreas Wold Sløgedal & Markus Dybevold Simonsen

June 12, 2026

Abstract

This report presents our work on Project 3 in INF265, where we implemented and evaluated transformer-based models for two different tasks. In Part I, we developed an encoder-only transformer for sentiment classification on IMDb movie reviews, including a custom implementation of masked multi-head self-attention in PyTorch. The model achieved an accuracy of 84.88%, demonstrating that the architecture was able to capture useful contextual information for distinguishing positive and negative reviews. In Part II, we implemented a decoder-only transformer for autoregressive text generation on a subset of the GooAQ dataset. We analyze the training behaviour, evaluate generated outputs, and examine the impact of different sampling strategies on model performance.

Contents

1	Introduction	3
2	Part I: Encoder-only Model for Sentiment Classification	3
2.1	Problem Setup & Data Exploration	3
2.1.1	Problem Formulation	3
2.1.2	Dataset & Preprocessing	3
2.1.3	Exploratory Analysis	4
2.2	Model Design & Training	5
2.2.1	Model Architecture	5
2.2.2	Loss Function	6
2.2.3	Training Procedure	6
2.3	Results & Discussion	6
2.3.1	Results	6
2.3.2	Discussion	8
2.3.3	Conclusion	10
3	Part II: Decoder-only Model for Text Generation	10
3.1	Task overview	10
3.2	Dataset and Tokenizer	10
3.3	Model Architecture	11
3.4	Training setup	12
3.5	Results	12
3.6	Sampling strategies	13
3.7	Discussion	15
3.8	Conclusion	15
A	Division of labor	16
B	Note on use of Artificial Intelligence	16

1 Introduction

Transformers are a central model family in modern natural language processing. In this project, we worked with two transformer variants: an encoder-only model for sentiment classification and a decoder-only model for text generation.

The goal of Part I was to understand how masked multi-head self-attention works in practice by implementing core components from scratch and training a sentiment classifier on IMDb movie reviews. The goal of Part II was to study autoregressive text generation by implementing and training a decoder-only transformer on question-answer data.

In this report, we describe our implementations, training procedures, results, and the main challenges we encountered.

2 Part I: Encoder-only Model for Sentiment Classification

2.1 Problem Setup & Data Exploration

2.1.1 Problem Formulation

In the first part of this project, we implement an encoder-only transformer model for sentiment classification using a dataset of IMDb movie reviews [2]. The objective of this task is to design a model that takes a tokenized sequence of words as input and predicts the corresponding sentiment label, which is positive or negative. This task aims to show how the masked attention mechanism operates within an encoder architecture, and how it can be effectively applied for the task of sentiment classification.

2.1.2 Dataset & Preprocessing

Initially the dataset is divided into a training-, validation- and test set, with size 25,000, 5,000 and 20,000 respectively. A seed is also set to ensure reproducibility. From this it is important to ensure that the datasets have loaded properly and are correctly formatted. After loading the reviews all HTML tags are removed along with any special characters. After this the text is converted to lowercase.

Following from the lowercase conversion, each review is tokenized. This means that each review is converted from a string to a sequence of integers. This is done by splitting the words on whitespace, so that the text is broken into word like pieces separated by space. From this a vocabulary is built mapping words to integer IDs, like for example:

"movie" → 20
"great" → 90
"bad" → 84

During this process, only the words with a minimum frequency is kept. This is decided by the `MIN_FREQUENCY` parameter, ensuring only tokens appearing a minimum of 10 times in the dataset are included. In addition, the vocabulary size is limited by the `VOCAB_SIZE` parameter, which restricts the vocabulary to the 10,000 most frequent tokens. This is useful both for reducing the model size, in addition these measures speeds up training along with avoiding overfitting on special words.

Additionally, a set of special tokens is introduced: `[PAD]` (padding token), `[UNK]` (unknown token), and `[CLS]` (classification token). These tokens are used to handle input sequences in a consistent manner. The `[PAD]` token is used to pad shorter sequences so that all inputs have the same length. This is controlled by the `MAX_LENGTH` parameter, which defines that each review

is set to a fixed length of 256 tokens. Reviews longer than this length are truncated, while shorter ones are padded. The unknown, [UNK], token represents words that are not part of the constructed vocabulary. This could be due to the token not being observed during training or due to low frequency. The [CLS] token is prepended to each input sequence and will serve as a global representation of the entire sequence. Its final embedding will be used as input to the classifier to predict the sentiment of the review.

The final data loader retrieves samples from the dataset, fits them into batches, and performs and carries out a shuffle on the batch. This is done to improve the stability and efficiency of the optimization process, as batching enables parallel computation while shuffling reduces the risk of the model learning patterns from the ordering of the data, which clearly applies to the training set.

2.1.3 Exploratory Analysis

Before, during, and after preprocessing, exploratory data analysis is performed. This helps to understand the structure and characteristics of the data prior to model development by identifying patterns, anomalies, and potential imbalances, which can inform preprocessing decisions and model design.

To begin with, the balancing of the classes was inspected. This clearly showed that the classes were evenly distributed. This indicates that no adjustments were required to account for class imbalance during training. In addition, the dataset was inspected for empty reviews and missing labels, none were found, no measures were thus taken to control for missing values.

An excerpt of the training set is also printed, both before and after preprocessing. This allows for a clear comparison of how a raw review is transformed through the pipeline. This step illustrates how the original text input is mapped into a numerical representation suitable for the model.

After inspecting the tokenization process, we analyzed the distribution of tokenized sequence lengths to investigate the maximum sequence length. The distribution shows how long the input reviews typically are after tokenization, making it possible to balance two competing concerns: preserving information from longer reviews by limiting truncation, and reducing unnecessary computation caused by padding shorter reviews.

From Figure 1, we observe that the median sequence length is 191. Together with the truncation rate of 33.4% at a maximum sequence length of 256, as shown in Figure 2, indicates that the chosen maximum sequence length of 256 is appropriate, as it captures the majority of the data while limiting the amount of truncation applied to longer sequences.

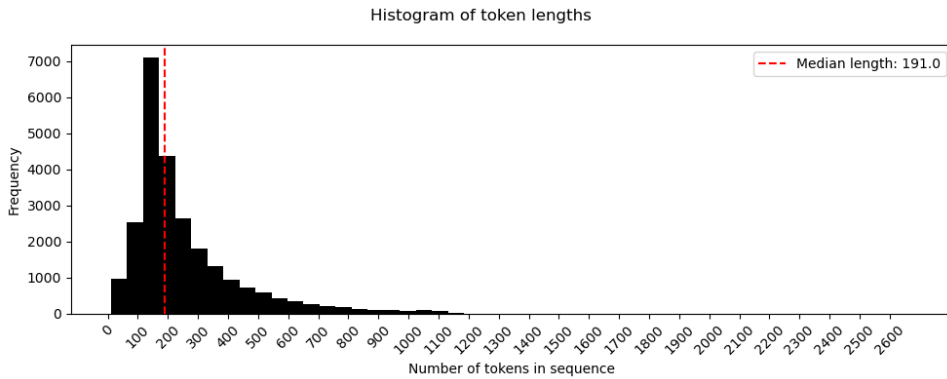


Figure 1: Distribution of sequence lengths in the training set.

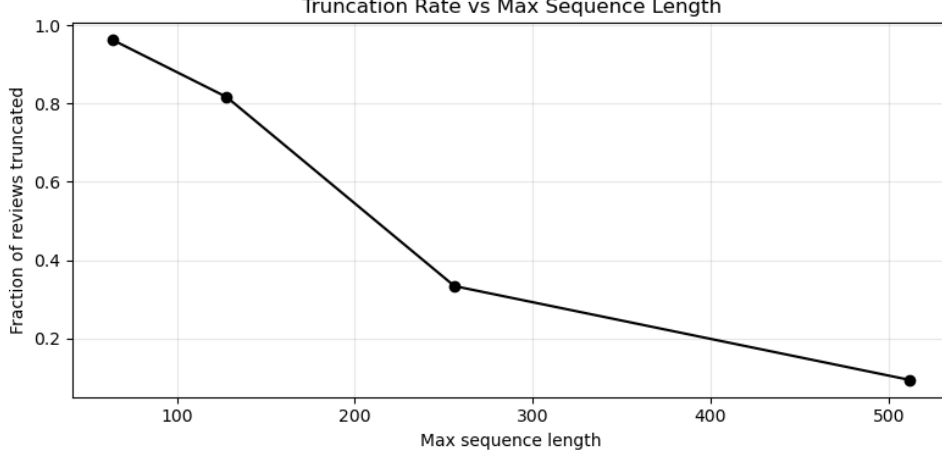


Figure 2: Truncation rate v max sequence length.

Additional exploratory analysis was conducted by inspecting the most and least frequent words, cumulative vocabulary coverage, and token frequency by vocabulary rank. This helped verify that the vocabulary captured the main structure of the dataset, while also revealing whether rare tokens, noisy terms, or unusual frequency patterns might affect preprocessing or model performance. Overall, these analyses did not reveal any significant irregularities or patterns that required further adjustments to the preprocessing pipeline or model design.

2.2 Model Design & Training

2.2.1 Model Architecture

For the sentiment classification task, an encoder-only transformer was implemented to map each IMDb review to a single binary prediction. This is done by mapping each token ID to a vector representation through an embedding layer. In addition, as the model do not inherently encode word order, positional encodings are added to the token embeddings so that the model capture where each token appears in the sequence.

The resulting sequence of embeddings is then passed through several encoder blocks implemented using the `EncoderBlock` class. In each block, multi-head self-attention allows every token to attend to the other tokens in the review, making it possible to capture contextual relationships between words. This is followed by a position-wise feed-forward network that further transforms the token representations. After both the attention and feed-forward sublayers, residual connections, layer normalization, and dropout are applied to improve training stability and reduce overfitting. By stacking multiple encoder blocks, the model can gradually build richer and more informative representations of the input sequence.

Finally, the special `[CLS]` token is used as a summary representation of the entire review. Through the self-attention mechanism in each encoder block, this token is updated using information from all other tokens in the sequence. After the final encoder layer, the representation corresponding to the `[CLS]` token is extracted and passed through a linear classification layer. This produces a single logit, denoted by z , which is then converted into a probability using the sigmoid activation function, as shown in Equation 1. The resulting value $\hat{y}$ represents the model’s predicted probability that the review belongs to the positive sentiment class.

$$\hat{y} = \sigma(z) = \frac{1}{1 + e^{-z}} \quad (1)$$

The sigmoid function maps any real-valued input to a value between 0 and 1, making it suitable for binary classification. Values close to 1 indicate a strong prediction of positive sentiment, while values close to 0 indicate negative sentiment. During evaluation, this probability can be converted into a class prediction by applying a threshold, typically 0.5.

In this implementation, each token is represented by a 96-dimensional vector after the embedding layer. A larger embedding dimension gives the model greater capacity to encode semantic and contextual information, but also increases the number of parameters and computational cost. The self-attention mechanism uses 4 attention heads, meaning that each encoder block can attend to the sequence from multiple representation subspaces in parallel. This allows the model to capture different types of relationships between tokens, rather than relying on a single attention pattern. Finally, the model consists of 3 stacked encoder blocks, which allow the token representations to be progressively refined and enable the model to learn more complex contextual patterns across the review.

2.2.2 Loss Function

Given that this is a binary classification task, a binary cross-entropy loss function was used. For a single training example, the binary cross-entropy loss is defined as shown in Equation 2:

$$\mathcal{L} = -(y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})), \quad (2)$$

where $y \in \{0, 1\}$ is the true sentiment label and $\hat{y}$ is the predicted probability of the positive class. The loss compares the predicted probability with the ground truth label. This function strongly penalizes confident incorrect predictions, and similarly encourages the model to assign a high probability to the correct class.

2.2.3 Training Procedure

The model was trained using mini-batches from the training set, while performance on the validation set was evaluated after each epoch. In the implementation, a batch size of 64 was used, and the training data were shuffled before each run through the `DataLoader`. For each batch, the token sequences and labels were moved to the selected device, and an attention mask was created from the padding tokens so that the model would not attend to padded positions.

The sequences were then passed through the model to obtain the predicted probabilities, and the binary cross-entropy loss was computed with respect to the true labels. The gradients were backpropagated through the network, after which gradient clipping with a maximum norm of 10.0 was applied to improve training stability. The model parameters were updated using the AdamW optimizer with a learning rate of 10^{-3} and weight decay of 10^{-3} . Training was carried out for 3 epochs, and after each epoch, the model was evaluated on the validation set in evaluation mode without gradient computation. During both training and validation, predictions above 0.5 were interpreted as positive sentiment when computing classification accuracy.

As specified in the project description, no hyperparameter tuning was performed due to the computational cost of the task. Therefore, the provided hyperparameter and model configurations was used, as described above.

2.3 Results & Discussion

2.3.1 Results

To investigate training performance, the model was evaluated after each epoch by calculating both loss and accuracy. These metrics were tracked for the training and validation sets to assess

how well the model learned from the training data and how effectively it generalized to unseen examples. Monitoring the loss also made it possible to detect slow convergence or poor learning behavior early, which is especially important for this task due to its high computational cost. This allowed training to be stopped if the model did not perform as expected. The training and validation metrics are shown in Table 1, and the corresponding loss curves are plotted in Figure 3. Both the training and validation loss decrease quickly at first, followed by a clearly stabilization between the second and third epoch. This trend is especially visible in the validation loss, which begins to flatten over this interval. Although the curves do not completely converge, the reduced improvement toward the end of training suggests that further training would likely yield only modest gains relative to the additional computational cost.

Table 1: Part I results summary.

Epoch	Train. Loss	Train. Accuracy	Val. Loss	Val. Accuracy
1	0.5522	0.7036	0.4227	0.8084
2	0.3733	0.8344	0.3758	0.8370
3	0.3015	0.8740	0.3531	0.8538

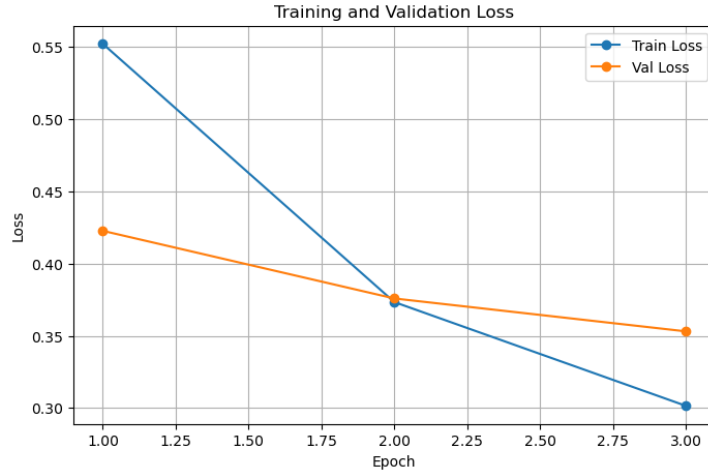


Figure 3: Training- & Validation Loss.

To obtain an unbiased estimate of the model’s generalization performance, the final model was evaluated on the unseen test set. It achieved a test accuracy of **0.8488**, which is consistent with the performance observed on the training and validation sets. This suggests that the model generalizes reasonably well to unseen data and does not show strong signs of overfitting. The model also achieved a reasonably equal accuracy on both of the sentiment labels, as shown in the normalized confusion matrix in Figure 4. The probability distribution shown in Figure 5 also supports this observation.

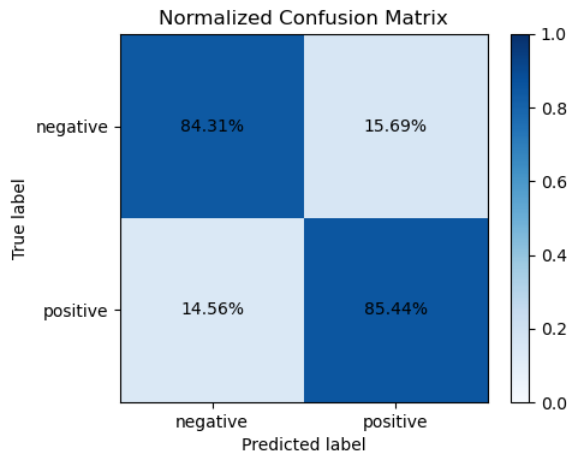


Figure 4: Normalized confusion matrix.

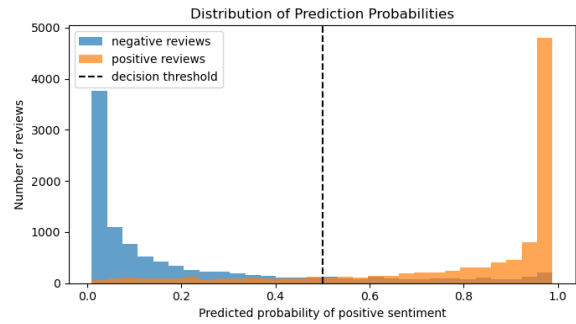


Figure 5: Predicted probabilities distribution.

2.3.2 Discussion

As described above, the results indicate that the encoder-based model was able to learn meaningful sentiment patterns from the IMDb reviews, while also generalizing well, without any strong signs of overfitting. The model did also not seem to prioritize any of the sentiment classes. Still the model has room to improvement, as it fails to correctly label approximately 15% of the reviews.

For the negative reviews, one possible explanation for a wrong classification could be the inclusion of positively loaded words when discussing specific parts or details of the movie. This is clearly illustrated in **Failed Review #1**.

True label: Negative
Predicted label: Positive
Model output: 0.9871

“Although this is not a great movie, it has its interesting moments. [...] The location is absolutely stunning and spectacular. It is beautiful, [...] wonderful character actors [...] He is literally hysterical [...] she is great, as always [...] she has always been a great character actress [...]”

Although the overall sentiment of this review is negative, it contains several positive expressions, such as “absolutely stunning”, “spectacular”, “wonderful”, and “she is great, as always”. The excerpt therefore contains more explicit positive descriptions than negative ones, which may explain why the model classified it as positive. This suggests that the model relied heavily on local sentiment-bearing words, while failing to capture the broader context and overall negative judgement of the review.

Another interesting example is shown in **Failed Review #8**.

True label: Positive
Predicted label: Negative
Model output: 0.1275

“this bw film reached the movie house of my village about 18 months after its release [...] without the nostalgia it would probably be just another comedy . but then , another movie that once had me in even more , the spike jones outing save my child , now seems dated and stilted apart from some orchestra bits and weaver scenes [...] to me this one at least rates eight out of ten .”

This review contains positive sentiment overall, but the positive descriptions are relatively mild compared to the stronger sentiment words seen in other examples. Phrases such as “without the nostalgia it would probably be just another comedy” and “to me this one at least rates eight out of ten” indicate a positive opinion, but they are not particularly enthusiastic. This suggests that the model may struggle when sentiment is expressed in a more moderate or understated way.

To further investigate the model’s performance, it was tested on a set of externally chosen IMDb reviews, as shown in Table 2. The reviews were selected from different genres to examine whether any differences in performance could be observed across review types. However, the model managed to classify all of the selected reviews correctly, suggesting that it was able to generalize reasonably well beyond the original test set. It should still be noted that this evaluation was based on only a small number of manually selected reviews, and therefore cannot be considered a comprehensive measure of performance.

Table 2: Sentiment predictions on external movie reviews.

Review excerpt	Sentiment	Model prediction	Model output
“From the prestige Asylum Films...”	Negative	Negative	0.0332
“10/10 for my friend Chicken Joe...”	Positive	Positive	0.9224
“I recently saw <i>The Bridge on the River Kwai</i> ...”	Positive	Positive	0.9194
“Like the child – and adults...”	Positive	Positive	0.8614

Note: The excerpts were retrieved from IMDb reviews for the movies *Sharknado*, *Surf’s Up*, *The Bridge on the River Kwai*, and *The Babadook*.

Even though the model performs reasonably well, it has some clear limitations. Since the model was trained from scratch using a word-level tokenizer, rare words are mapped to the unknown token. This may remove useful sentiment information from the input, especially when important opinion-bearing words are not included in the vocabulary. In addition, reviews that exceed the maximum sequence length are truncated, meaning that parts of the review that are relevant for determining the overall sentiment may be lost. Furthermore, no hyperparameter tuning was performed, as specified in the project description, meaning that the chosen model configuration may not be optimal. For the same reason the model was also trained for only 3 epochs, which may limit its ability to fully learn more complex sentiment patterns.

Based on the observed model performance and its limitations, several measures could be taken to improve performance. First, hyperparameter tuning could be performed to find better values for parameters such as the learning rate, weight decay, dropout rate, embedding dimension, number of attention heads, and number of encoder layers. Second, using a subword tokenizer instead of a word-level tokenizer could reduce the number of unknown tokens and allow the model to handle rare or unseen words more effectively. Third, increasing the maximum sequence length could help preserve more context from longer reviews. All of these measures would likely improve performance, but at the cost of greater model complexity and increased computational requirements. Hence, there is a clear trade-off between potential performance improvements and computational efficiency. For this project, the provided configuration represents a reasonable balance between performance, training time, and implementation complexity.

Another possible extension to the model could be the inclusion of film-specific metadata, such as genre, recommended age rating, movie format, or year of release. For example, in the case of genre, sentiment-bearing words may have different meanings depending on the type of movie

being reviewed. For example, words such as “violent” or “disturbing” may be positive in horror or action reviews, but negative in other genres. Whether or not these implementations would yield increased model performance is not certain. However, such an investigation could reveal whether the model performs differently across movie categories and whether incorporating genre-specific context improves sentiment classification. This may, however, also limit the model’s generalizability beyond film reviews.

2.3.3 Conclusion

Overall, the encoder-based sentiment classifier achieved reasonable performance for this classification task. With a test accuracy of **0.8488**. The training and validation results indicate that the model learned meaningful sentiment patterns, and generalized well to unseen data, and showed little to no sign of overfitting. The model also seem to treat both classes equally, with approximately the same accuracy on the two classes. Further inspection indicated that the model struggled with mixed or moderately expressed sentiment. Despite these limitations, the model was able to classify both the test set and the selected external reviews with satisfactory performance. Although the model has clear limitations and several areas for improvement, and currently unsatisfactory performance, to serve as a standalone sentiment classifier, it may still provide a solid foundation for a more advanced sentiment classification model.

3 Part II: Decoder-only Model for Text Generation

3.1 Task overview

In this part, we built a decoder-only transformer model for question answering using a subset of the GooAQ dataset, which consists of question–answer pairs [1]. The goal was to create a model that can generate an answer to a given question.

The model works autoregressively, meaning it generates text step by step based on what has already been generated. During training, each example contains both a question and its corresponding answer, which helps the model learn how answers typically follow from questions. At inference time, only the question is provided, and the model generates an answer until it reaches a stopping point.

3.2 Dataset and Tokenizer

Each question-answer example from the dataset is turned into a single sequence of the form:

question + [SEP] + answer + [END]

This is done in `QADataset`, where the [SEP] token is used to separate the question from the answer, and the [END] token marks where the sequence should stop.

To train the model, we created a shifted version of this sequence so that it learned to predict what comes next at each step. This means that the input and target sequences are almost identical, but the target is shifted by one position. This allows the model to learn to predict the next token at each step. In `dataset.py`, the sequence is first padded or truncated to length `max_len + 1`, and then split into source and target so the shifting works correctly. Padding positions in the target are set to -100, which means they are ignored by the cross-entropy loss during training.

We used four unique tokens: [UNK], [SEP], [END], and [PAD]. For unknown text we used [UNK], [SEP] separates the question from the answer, [END] tells the model where the sequence should stop, and [PAD] is used to make all sequences the same length. Our configuration explicitly says that the maximum sequence length is 128 tokens.

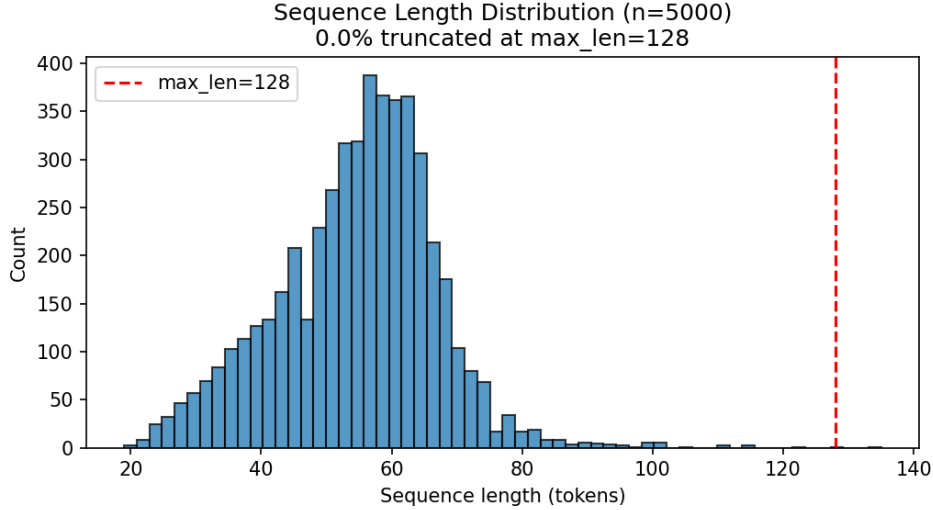


Figure 6: Distribution of sequence lengths in the dataset. The dashed line shows the maximum sequence length of 128 tokens.

For tokenization, we used a Byte-Pair Encoding (BPE) tokenizer trained on the dataset. In `tokenizer.py`, the tokenizer is built with a vocabulary size of 20,000 and trained with the four special tokens. The reason BPE is preferred is because it handles rare words much better than word-level tokenization. Instead of treating an unseen word as unknown, we first try to split it into smaller subwords because this makes the model more flexible and helps reduce dependence on a single [UNK] token. The reason for this is that unknown words can be made up of subwords.

3.3 Model Architecture

The model is a decoder-only transformer implemented in `model.py`. It consists of an embedding layer, positional encoding, a stack of decoder blocks, and a final output layer.

Each input token is first mapped to a vector using an embedding layer (`nn.Embedding`) with a vocabulary size of 20,000 and an embedding dimension of 512. Since the model does not understand the order of tokens, we add positional encodings to these embeddings. These encodings provide information about the position of each token in the sequence, and in our implementation they are precomputed and added during the forward pass.

The main part of the model is a stack of 5 identical decoder blocks, implemented as `DecoderBlock`. Each block applies layer normalization before each sublayer, and residual connections are used throughout. Inside each block, we first apply masked multi-head self-attention using `nn.MultiheadAttention` with 8 heads. Both a causal mask and a padding mask are used to control which tokens the model can attend to. This is followed by a feed-forward network with two linear layers and a GELU activation. Dropout with probability 0.1 is applied after each of these components.

Figure 7 shows an example of the attention pattern learned by the model.

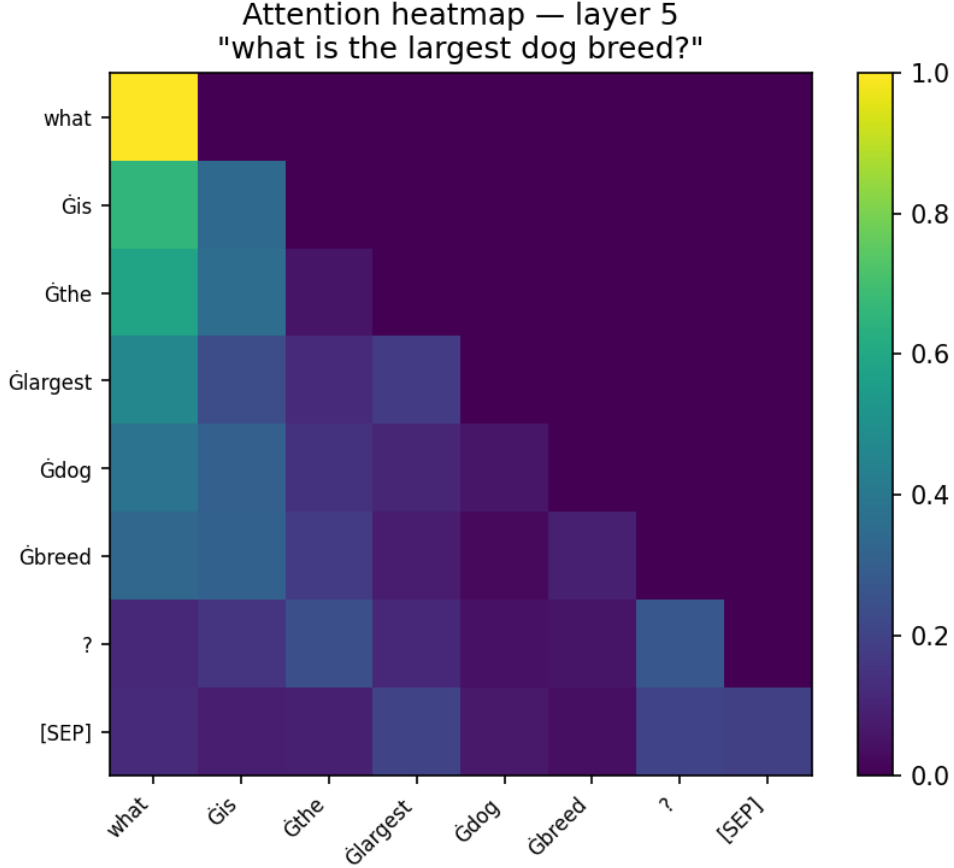


Figure 7: Attention heatmap from the final decoder layer showing causal attention over previous tokens.

The causal mask itself is created in `generate_causal_mask` in `model.py`, and ensures that the model can only attend to previous tokens when making predictions. Finally, after all decoder blocks, a linear layer maps the hidden representations to the vocabulary size, producing logits for each possible next token. The model has roughly 36 million trainable parameters in total.

3.4 Training setup

The decoder-only model was trained for 5 epochs with a batch size of 128 using the AdamW optimizer and a learning rate of $1e-4$. The training objective is cross-entropy loss, which measures how well the model predicts the correct next token in the sequence.

The training was done on GPU. Furthermore, the model has approximately 36 million trainable parameters. During training, checkpoints were saved every 500 batches and at the end of each epoch. The idea is that training could be resumed at a checkpoint if the interruptions occurred.

3.5 Results

Figure 8 shows the training loss over 5 epochs. The loss decreases rapidly from approximately 4.7 in the first epoch to 3.9 in the second, and then continues to decrease more gradually, reaching around 3.2 by epoch 5. This behaviour is expected, as the model first learns basic token patterns and then gradually captures more complex relationships. The curve begins to flatten towards the final epoch, but does not fully converge, suggesting that additional training could further improve performance.

To evaluate the model, we generated answers to several questions using both greedy sampling

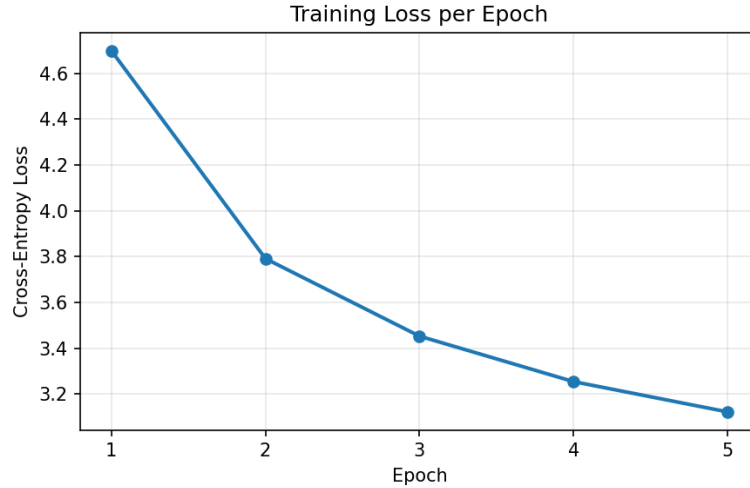


Figure 8: Cross-entropy training loss per epoch.

and top- p sampling (with temperature = 0.7 and $p = 0.95$). We also created a Streamlit application to interactively test the chatbot.

From our experiments, greedy sampling produces safe but often repetitive outputs. In many cases, it repeats phrases or parts of the question instead of providing a meaningful answer. This results in reduced diversity and less informative responses.

Top- p sampling, on the other hand, produces more diverse outputs. The generated text is often more natural, but can also include inconsistencies or incorrect information. This reflects a trade-off between reliability and diversity in the generated text.

Below are some example outputs generated by the model:

Question: What is the largest dog breed?

Greedy:

the largest dog breed of dog breed is the largest breed of dog breed. the breed is the breed of dog breed. the breed is the only breed of dog breed.

Top-p:

the median male dog breed is a breeding dog breed with two female dog breeds: males, males, females, and males. males are known for their domestic dog breeds, but females are often preferred for their dog breeds.

Question: What is the capital of Norway?

Greedy:

the capital of norway is the capital of norway. it is the capital of the republic of norway. it is the capital of the republic of denmark, the capital of luxembourg, and the capital of luxembourg.

Top-p:

the capital of norway consists of the three branches: the central government, the capital, and the capital.

3.6 Sampling strategies

In this project, we implement two sampling strategies to generate text from the model. Greedy sampling selects the token with the highest probability at each step. This results in a determin-

istic approach, meaning that, in theory, the same input will always produce the same output. From our experience, greedy sampling produces repetitive and less informative responses, since the model always follows the most likely continuation.

Top- p sampling, on the other hand, selects tokens from the smallest subset whose cumulative probability exceeds a threshold p . Before sampling, the logits are scaled using a temperature parameter, which controls how sharp or flat the probability distribution is. In our experiments, we use $p = 0.95$ and a temperature of 0.7.

The choice of p and temperature values has a direct effect on the generated output. A higher value of p includes more tokens in the candidate set, increasing diversity but also the risk of incorrect outputs. A lower value of p restricts the selection to only the most likely tokens, resulting in more conservative and often repetitive text. Similarly, a higher temperature makes the model more random and produces more varied outputs, while a lower temperature makes the model more focused and predictable.

Figure 9 illustrates how the probability mass is distributed across possible next tokens for a sample input. The distribution is highly skewed, with one token dominating most of the probability mass. This helps explain why greedy sampling often produces repetitive outputs, while top- p sampling can introduce more variation by allowing less probable tokens to be selected.

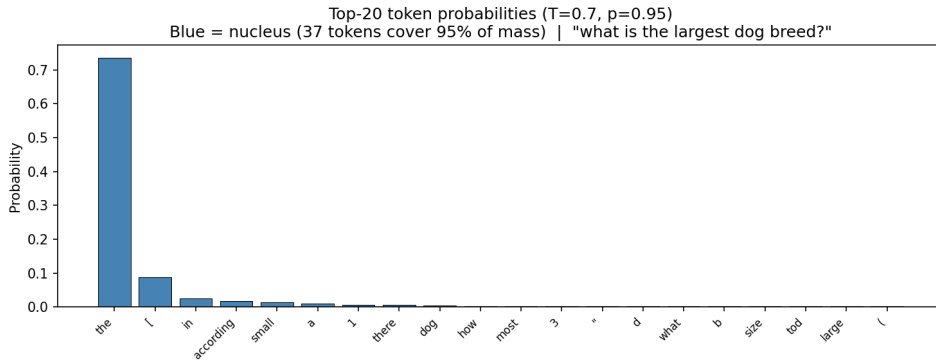


Figure 9: Top- p token probability distribution with nucleus covering $p = 0.95$.

These effects can be observed in practice. With a lower temperature (temperature = 0.7), the model produces more structured and predictable text, even though the answer may still be repetitive and not very informative. In contrast, when the temperature is increased significantly (temperature = 1.9), the output becomes highly unstable and loses its structure completely, resulting in incoherent and almost random text.

To illustrate the effect of temperature, we use the same question as in Section 3.5: “What is the largest dog breed?”

Top- p ($\tau = 0.7$, $p = 0.95$):

“the largest dog in the world is the largest dog in the world ...”

Top- p ($\tau = 1.9$, $p = 0.95$):

“tti descent0000 to 17 82 ba children’s crisp ever...”

These differences explain the model’s behaviour observed in the results section. Greedy sampling tends to produce repetitive and less informative outputs, while top- p sampling generates more diverse responses at the cost of occasional inconsistencies. The choice of temperature and top- p parameters plays an important role in balancing this trade-off, as they directly control the level of randomness in the generated text. From the example above, it is evident that increasing the

temperature (e.g., to 1.9) causes the output to break down completely, producing unstable and essentially meaningless text.

3.7 Discussion

The results show that the model is able to learn the overall structure of question–answer pairs from the dataset and generate grammatically correct responses. However, the quality of the generated answers is inconsistent, and several clear limitations become apparent when evaluating the model in practice.

The main issue is hallucination. The model often generates answers that sound reasonable, but are factually incorrect. For example, when asked about the capital of Norway, the model produces answers that are unrelated to the question. This is expected, since the model does not store factual knowledge, but instead learns statistical patterns from the training data.

Another limitation is repetition. The model frequently repeats phrases and struggles to produce meaningful continuations, especially when using greedy sampling. While top- p sampling reduces this behaviour by introducing more variation, it can instead lead to inconsistent and incoherent outputs.

Another challenge is the lack of a clear evaluation metric for the chatbot. Unlike classification tasks where performance is measured using accuracy, evaluating generative models is more difficult. The training loss only measures how well the model predicts the next token, but a low loss here does not necessarily mean that the model is great at generating accurate and meaningful answers. Therefore, evaluating the chatbot is based on judging generated output, which is subjective and makes it harder to assess consistently.

Based on the training curve shown in Figure 8, the model does not appear to have fully converged, suggesting that underfitting is more likely than overfitting, although it is difficult to conclude this with certainty without a validation set. This indicates that the model could benefit from further training. Training for more epochs would likely allow the model to converge further and improve performance. Increasing the model size and allowing longer input sequences could help capture more complex patterns. In addition, introducing a validation set would make it easier to monitor training and detect overfitting or underfitting. Finally, experimenting with alternative decoding methods, such as beam search, could potentially produce more coherent outputs compared to greedy sampling, while avoiding some of the randomness of top- p sampling.

3.8 Conclusion

In this part, we implemented a decoder-only transformer model. The model is able to learn the structure of question–answer pairs and generate grammatically correct responses. However, the results show clear limitations. The model often produces hallucinations and repetitive outputs, highlighting the difficulty of generating meaningful text without access to factual knowledge. Furthermore, the comparison between greedy and top- p sampling shows that the choice of decoding strategy has a significant impact on the generated output. Greedy sampling produces more predictable but repetitive responses, while top- p sampling generates more diverse outputs at the cost of incoherence.

Overall, this part of the project has given us a practical understanding of how decoder-only models generate text, how training loss relates to model behaviour, and why evaluating generative models is more challenging than standard classification tasks.

List of Figures

1	Distribution of sequence lengths in the training set.	4
2	Truncation rate v max sequence length.	5
3	Training- & Validation Loss.	7
4	Normalized confusion matrix.	8
5	Predicted probabilities distribution.	8
6	Distribution of sequence lengths in the dataset. The dashed line shows the maximum sequence length of 128 tokens.	11
7	Attention heatmap from the final decoder layer showing causal attention over previous tokens.	12
8	Cross-entropy training loss per epoch.	13
9	Top- p token probability distribution with nucleus covering $p = 0.95$	14

References

- [1] Daniel Khashabi, Sewon Min Ng, Tushar Khot, Ashish Sabharwal, Hannaneh Hajishirzi, and Luke Zettlemoyer. Gooaq: Open question answering with diverse answer types. *arXiv preprint arXiv:2104.08727*, 2021.
- [2] Andrew L. Maas, Raymond E. Daly, Peter T. Pham, Dan Huang, Andrew Y. Ng, and Christopher Potts. Learning word vectors for sentiment analysis. In Dekang Lin, Yuji Matsumoto, and Rada Mihalcea, editors, *Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics: Human Language Technologies*, pages 142–150, Portland, Oregon, USA, June 2011. Association for Computational Linguistics.

A Division of labor

Markus D. Simonsen focused on the encoder-only sentiment classifier (Part I), including model implementation, training, and evaluation. Andreas W. Sløgedal focused on the decoder-only chatbot (Part II), including model implementation, training, and evaluation. Both of us contributed to the overall report writing and reviewed each other’s work.

B Note on use of Artificial Intelligence

Following the faculty’s guidelines, artificial intelligence tools were used as support for this project. All decisions, implementations, and experiments were conducted and verified by the authors.

B: As Support for Language Editing and Improvement of Text

ChatGPT 5.2 (OpenAI) was used to improve grammar, readability, and clarity of the report. In addition, the tool was used to shorten and refine the wording. No conclusions were generated by the tool. All provided content and rephrasings were reviewed and edited before inclusion.

E: As Support for Programming

ChatGPT 5.2 (OpenAI) and Claude Opus 4.5 (Anthropic) were used as programming assistants during the development of the project code. The systems were used for code explanations, debugging, and suggestions. None of the tools were used to generate complete solutions; all code was written, tested, and validated by the authors.